CardioRisk-MLOps

Heart Disease Risk Classifier - End-to-End MLOps Pipeline

MLOps Assignment 1 (AIMLCZG523) - BITS Pilani, Work Integrated Learning Programmes **J Sai Anvesh Reddy** - BITS ID: 2025AE05969 **Repository:** https://github.com/SaiAnveshReddy/MLOps-Assignment

---

1. Objective and Problem Statement

The goal of this assignment is to design, build, and deploy a complete MLOps pipeline around a single classification problem: given a patient's clinical measurements, predict whether they are at risk of heart disease. The emphasis is less on squeezing out the last percentage point of accuracy and more on the *operational* surface around the model - experiment tracking, reproducible packaging, automated testing, CI/CD, containerization, orchestration, and monitoring.

The resulting system, which I've named **CardioRisk**, exposes a `/predict` REST endpoint that accepts a patient's raw clinical fields and returns a binary risk prediction with a confidence score.

2. Dataset

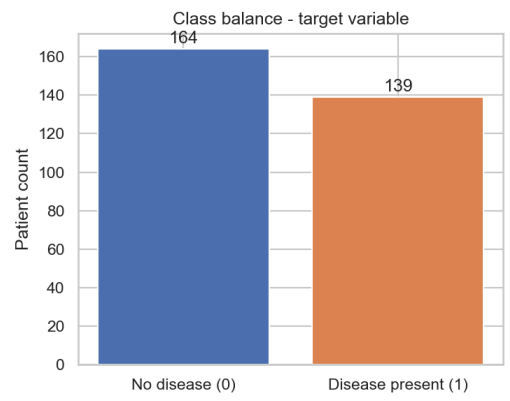
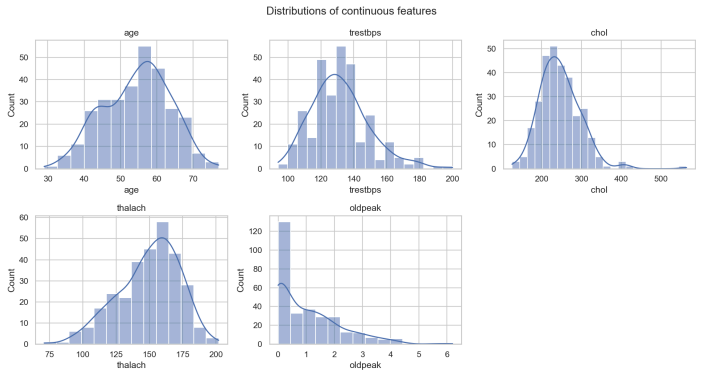
Heart Disease UCI Dataset - Cleveland subset (UCI Machine Learning Repository): 303 patient records, 13 clinical features + 1 target. Features: age, sex, chest pain type (`cp`), resting blood pressure (`trestbps`), cholesterol (`chol`), fasting blood sugar (`fbs`), resting ECG (`restecg`), max heart rate achieved (`thalach`), exercise-induced angina (`exang`), ST depression (`oldpeak`), ST slope (`slope`), major vessels (`ca`), and thalassemia (`thal`). The target (originally 0-4 disease severity) is collapsed to binary: 0 = no disease, 1 = disease present.

I chose the Cleveland subset specifically (rather than pooling it with the Hungarian/Switzerland/VA data UCI also distributes) because it is the only one of the four with near-complete records - the other three have far heavier missingness on `ca` and `thal`, which would have forced a much less defensible imputation strategy for a dataset this small. The raw file ships in the repository at `data/raw/processed.cleveland.data` so the project runs offline; `README.md` documents the `curl` command to re-download it from UCI.

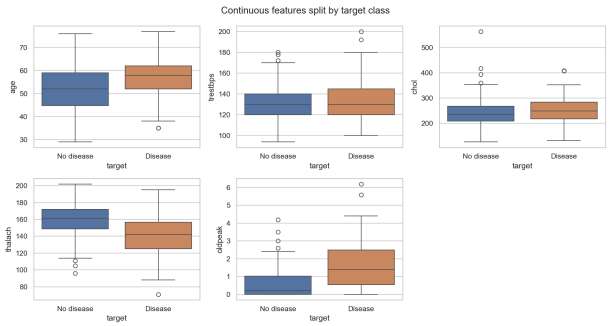
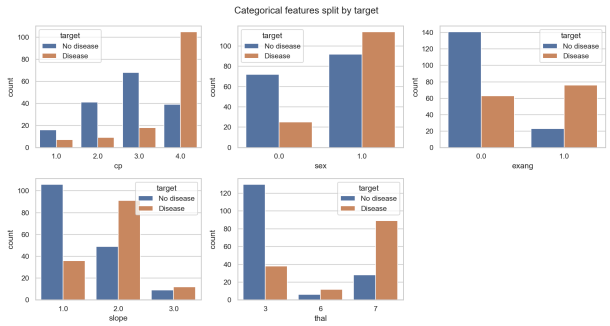
3. Task 1 - Data Acquisition & EDA

Full working notebook: `notebooks/01_eda.ipynb`.

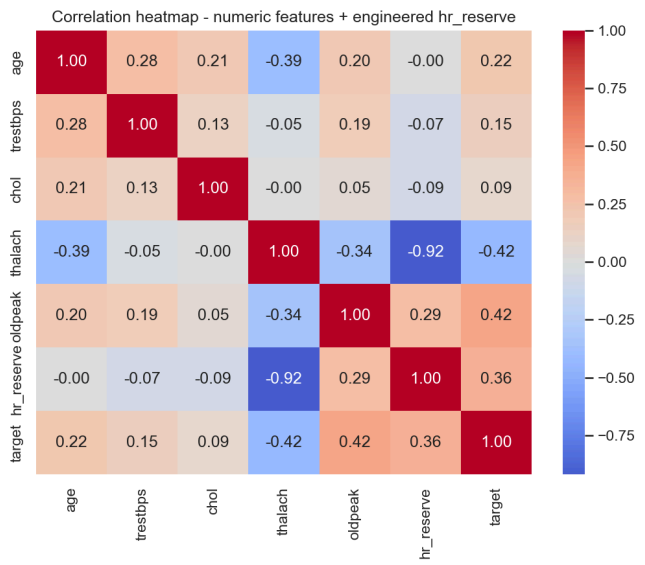
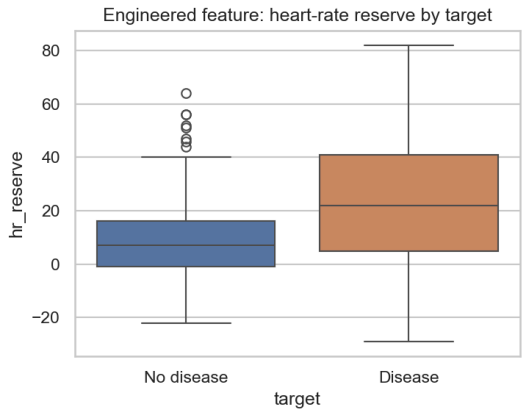
Cleaning. The raw file has no header row and encodes missing values as `?`. Only `ca` (4 missing) and `thal` (2 missing) are affected - 6 of 303 rows (~2%). Rather than drop these rows (the approach used in most public notebooks on this dataset), I mode-imputed both columns, since they are effectively categorical and the sample is already small enough that losing 2% of it isn't free (`heart_mlops.data.clean_data`).

Class balance	Numeric feature distributions
	

The split is roughly 54% no-disease / 46% disease - close enough to balanced that I used plain stratified splitting/CV rather than adding a resampler (SMOTE, class weighting), which would be unjustified complexity here. `chol` and `oldpeak` are right-skewed with a handful of high outliers; I judged these clinically plausible rather than data errors, so I kept them, but it's why `StandardScaler` is used in preprocessing.

Continuous features by target	Categorical features by target
	

`thalach` (max heart rate achieved) shows the clearest separation of any continuous feature - patients with disease reach a noticeably lower peak heart rate. Among categorical fields, `cp` and `thal` separate the classes most visibly; both are true multi-category (not ordinal) fields, so the pipeline one-hot encodes them rather than implying that chest-pain-type 3 sits "between" types 2 and 4.

Correlation heatmap	Engineered `hr_reserve` by target
	

No pair of raw features is strongly enough correlated to justify dropping one for multicollinearity (the strongest raw pair is `age` / `trestbps` at 0.28).

4. Task 2 - Feature Engineering & Model Development

4.1 Engineered feature: `hr_reserve`

Cardiology commonly estimates predicted maximum heart rate as `220 - age`; the gap between that and the achieved max heart rate (`thalach`) is the "heart rate reserve": `hr_reserve = (220 - age) - thalach`.

I want to be upfront about what this feature actually buys, rather than overstating it. From the heatmap above, `hr_reserve` correlates with `target` at **0.36**, which is honestly slightly *weaker* than `thalach` alone (**-0.42**) - and `hr_reserve` is almost perfectly anti-correlated with `thalach` (-0.92) by construction, so it isn't contributing independent signal. What it does provide is **interpretability**: a single clinically-named number instead of asking a model (or a report reader) to mentally recombine two raw columns. I kept it because it doesn't hurt model performance and makes the feature list easier to reason about - not because it statistically outperforms its source columns.

4.2 Preprocessing pipeline

Implemented in `heart_mlops.features.build_preprocessor`:

Feature group	Columns	Transform
Continuous	`age, trestbps, chol, thalach, oldpeak, ca, hr_reserve`	`StandardScaler`
Already-binary	`sex, fbs, exang`	passthrough
Multi-category	`cp, restecg, slope, thal`	`OneHotEncoder`

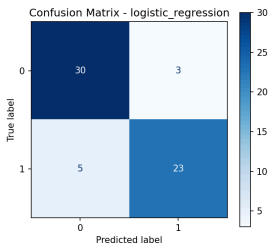
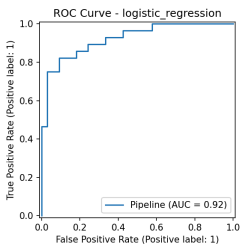
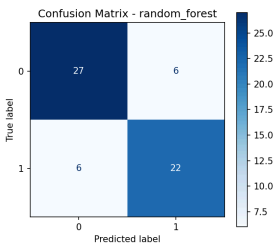
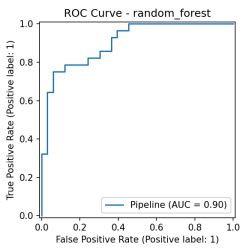
Preprocessing is wrapped into the same `sklearn.Pipeline` as the estimator (`build_full_pipeline`), so the saved model is self-contained - scaling and encoding happen automatically at inference time.

4.3 Models, tuning, and results

Two classifiers were tuned with `GridSearchCV` under 5-fold stratified cross-validation, scoring on ROC-AUC: **Logistic Regression** (`C in {0.1, 1.0, 3.0}`, L2) and **Random Forest** (`n_estimators in {150, 300}`, `max_depth in {4, 6, None}`, `min_samples_leaf in {1, 3}`). The grids were deliberately small: with 303 rows, an exhaustive search would mostly overfit the CV folds.

Model	CV mean ROC-AUC	Test accuracy	Test precision	Test recall	Test F1	Test ROC-AUC
Logistic Regression	0.9108	0.8689	0.8846	0.8214	0.8519	**0.9221**
Random Forest	0.8971	0.8033	0.7857	0.7857	0.7857	0.9037

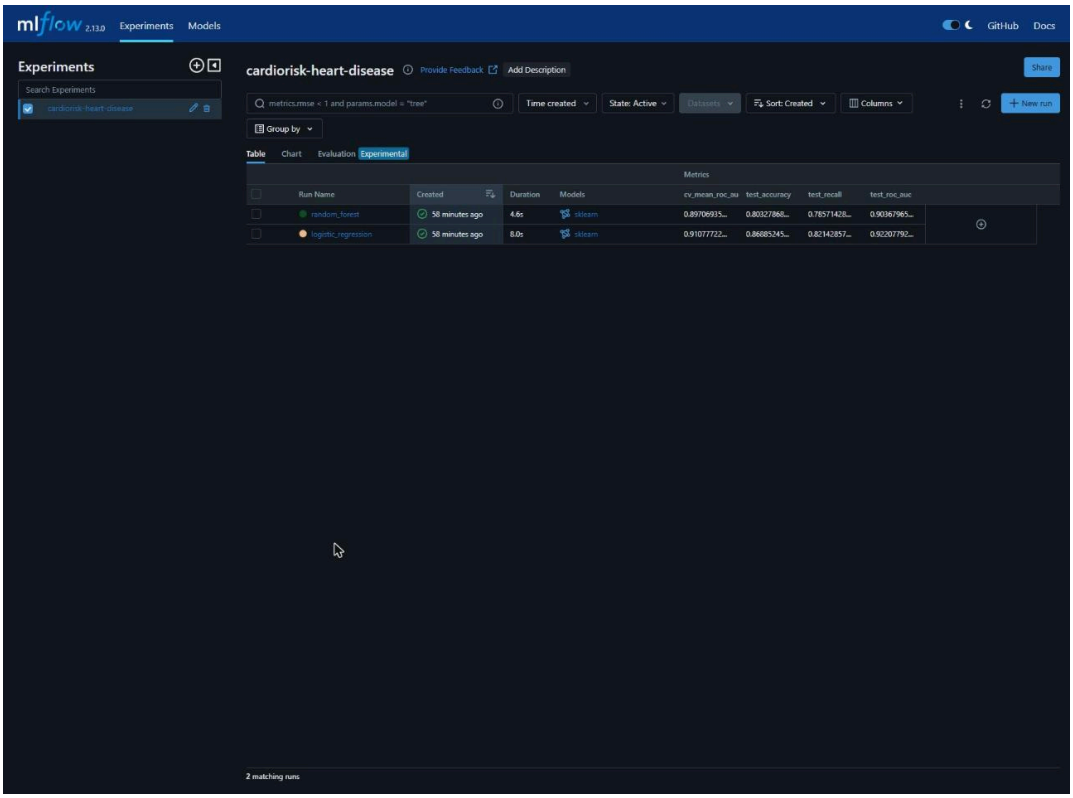
Selected model: Logistic Regression (`C=0.1`), on held-out test ROC-AUC. Full metrics are in `models/model_metadata.json`; the table and plots come directly from `python -m heart_mlops.train`.

LR confusion matrix	LR ROC curve	RF confusion matrix	RF ROC curve
			

Logistic Regression winning on a dataset this size (303 rows, 17 features after encoding) isn't surprising - tree ensembles generally need more data to beat a well-regularized linear model, and `C=0.1` (fairly strong L2 regularization) winning the grid search points the same way: the simpler, more constrained model generalizes better here.

5. Task 3 - Experiment Tracking (MLflow)

Every training run is logged to MLflow via `heart_mlops.train`, tracked locally under `mlruns/` (experiment: `cardiorisk-heart-disease`). Each run logs **parameters** (model type and `GridSearchCV.best_params_`), **metrics** (5-fold CV mean ROC-AUC plus test accuracy, precision, recall, F1, ROC-AUC) and **artifacts** (the confusion matrix and ROC curve PNGs above, plus the serialized sklearn pipeline via `mlflow.sklearn.log_model`). Runs can be inspected with `mlflow ui --backend-store-uri file:./mlruns`.



The screenshot is the live `mlflow ui` for this experiment with the metric columns enabled - the `test_roc_auc` values (0.9037 random_forest, 0.9221 logistic_regression) match Section 4.3 exactly.

6. Task 4 - Model Packaging & Reproducibility

- **Model format**: the winning pipeline (preprocessing + estimator) is serialized with `joblib` to `models/heart_risk_model.pkl`; the preprocessing step alone is also saved (`models/preprocess_pipeline.pkl`).
- **Dependency pinning**: `requirements.txt` pins exact versions of every dependency, so a fresh `pip install -r requirements.txt` reproduces the same environment.
- **Reproducible training**: a fixed random seed (`RANDOM_STATE = 5969`, the last 4 digits of my BITS ID) is threaded through the train/test split, CV folds, and both estimators.
- **Metadata**: `models/model_metadata.json` records the winning model, hyperparameters, CV/test metrics, exact feature column order, and seed - so the saved `.pkl` is never a mystery artifact.

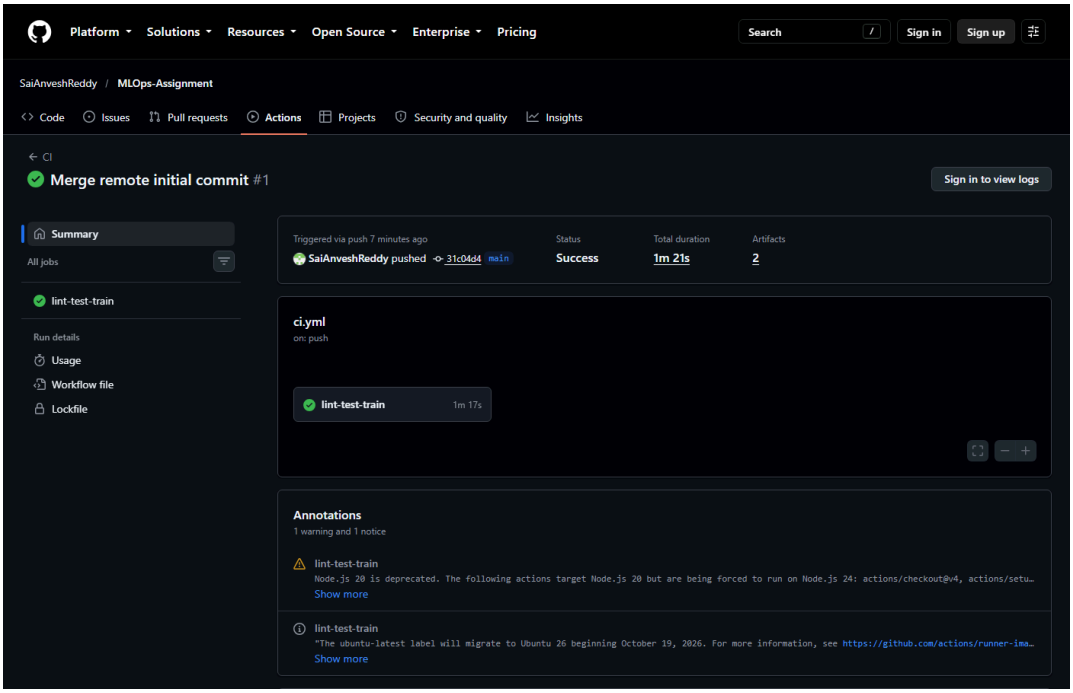
Because preprocessing is bundled inside the saved model, reproducing a prediction needs only `joblib.load("models/heart_risk_model.pkl").predict(df)`.

7. Task 5 - CI/CD Pipeline & Automated Testing

Unit tests: 17 Pytest tests across four files, all passing - `test_data.py` (loading, missing values, binary target), `test_features.py` (engineered feature arithmetic, pipeline fit/predict on real data), `test_inference.py` (`predict_risk()` output, missing-field errors) and `test_api.py` (`/health`, `/predict`, `/metrics`, HTTP 422 on invalid input). `tests/conftest.py` trains a small real pipeline into a temp path rather than mocking `predict_risk()`, so the tests exercise the actual inference code path.

GitHub Actions (`.github/workflows/ci.yml`) runs on every push/PR to `main`: (1) checkout + Python 3.11 with pip caching; (2) install dependencies; (3) **lint** with `flake8` - hard failure on real errors (syntax, undefined names), style issues soft-reported; (4) **test** with `pytest --junitxml`; (5) **train** with `python -m heart_mlops.train`, proving the pipeline reproduces from a clean checkout; (6) upload the test report and

trained model/MLflow runs as workflow artifacts. A failure at any step fails the job with the tool's native log output.



8. Task 6 - Model Containerization

`docker/Dockerfile` builds a `python:3.11-slim` image that installs `requirements.txt`, installs the `heart_mlops` / `api` packages, copies in the pre-trained `models/` artifacts, and serves the FastAPI app with `uvicorn` on port 8000. A `HEALTHCHECK` hits `/health` every 30s.

```
docker build -f docker/Dockerfile -t cardiorisk-api:local .
docker run -p 8000:8000 cardiorisk-api:local
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"age":57,"sex":1,"cp":4,"trestbps":140,"chol":289,"fbs":0,"restecg":2,
       "thalach":145,"exang":1,"oldpeak":2.0,"slope":2,"ca":1,"thal":7}'
# -> {"prediction":1,"risk_label":"high_risk","confidence":0.9283,...}
```

The image was built and run under Docker Desktop: the container reports `(healthy)` via the `HEALTHCHECK`, `/health` and `/predict` respond correctly (the sample patient comes back `high_risk` with 0.93 probability), and `docker logs` shows the structured JSON request logs.

Terminal - CardioRisk-MLOps: docker build & run

```
$ docker build -f docker/Dockerfile -t cardiorisk-api:local .
#4 [1/7] FROM docker.io/library/python:3.11-slim@sha256:e88e9763f943ec1834f992a4b51e0f24500486803e8bc534e5767af9ea65f6ce
#6 [2/7] WORKDIR /app
#7 [3/7] COPY requirements.txt pyproject.toml ./
#8 [4/7] RUN pip install --no-cache-dir -r requirements.txt
#9 [5/7] COPY src/ src/
#10 [6/7] COPY models/ models/
#11 [7/7] RUN pip install --no-cache-dir -e .
#12 naming to docker.io/library/cardiorisk-api:local done

$ docker images cardiorisk-api
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
cardiorisk-api:local   7034ca0b1bc7       1.59GB          353MB   U

$ docker run -d --name cardiorisk-api -p 8000:8000 cardiorisk-api:local
88621e367c3057aa0a0762ea8d1e1d103032c1fc447ef4f4ecde89a09242a710

$ docker ps --filter name=cardiorisk-api
CONTAINER ID   IMAGE                STATUS                   PORTS
88621e367c30   cardiorisk-api:local   Up 5 seconds (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

$ curl http://localhost:8000/health
{"status":"healthy"}

$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{"age":57,"sex":1,"cp":4,"trestbps":140,"chol":289,"fbs":0,"restecg":2,"thalac
h":145,"exang":1,"oldpeak":2.0,"slope":2,"ca":1,"thal":7}'
{"prediction":1,"risk_label":"high_risk","confidence":0.9283,"probability_disease":0.9283}

$ docker logs cardiorisk-api --tail 4
{"asctime": "2026-10-18 14:56:55,557", "name": "cardiorisk_api", "levelname": "INFO", "message": "request_handled", "method": "GET", "path": "/health", "status_cod
e": 200, "duration_ms": 0.84}
INFO:     172.17.0.1:38890 - "GET /health HTTP/1.1" 200 OK
{"asctime": "2026-10-18 14:56:55,668", "name": "cardiorisk_api", "levelname": "INFO", "message": "request_handled", "method": "POST", "path": "/predict", "status_c
ode": 200, "duration_ms": 66.17}
INFO:     172.17.0.1:38902 - "POST /predict HTTP/1.1" 200 OK
```

9. Task 7 - Production Deployment (Kubernetes / Minikube)

I chose **Minikube** over a public cloud cluster (GKE/EKS/AKS) - it's free, fully local, and explicitly listed as an acceptable target; a cloud cluster would add account/billing setup with no difference in the manifests themselves. `k8s/deployment.yaml` runs 2 replicas of `cardiorisk-api:local` with readiness/liveness probes on `/health` and modest CPU/memory requests/limits; `k8s/service.yaml` exposes it as a `LoadBalancer` Service on port 80 -> container port 8000 (step-by-step in `k8s/README.md`):

```
minikube start --driver=docker
minikube image load cardiorisk-api:local
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
kubectl get pods -l app=cardiorisk-api
minikube service cardiorisk-api-service --url
```

Terminal - CardioRisk-MLOps: Minikube deployment

```
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured


$ minikube image load cardiorisk-api:local
$ minikube image ls | grep cardiorisk
docker.io/library/cardiorisk-api:local

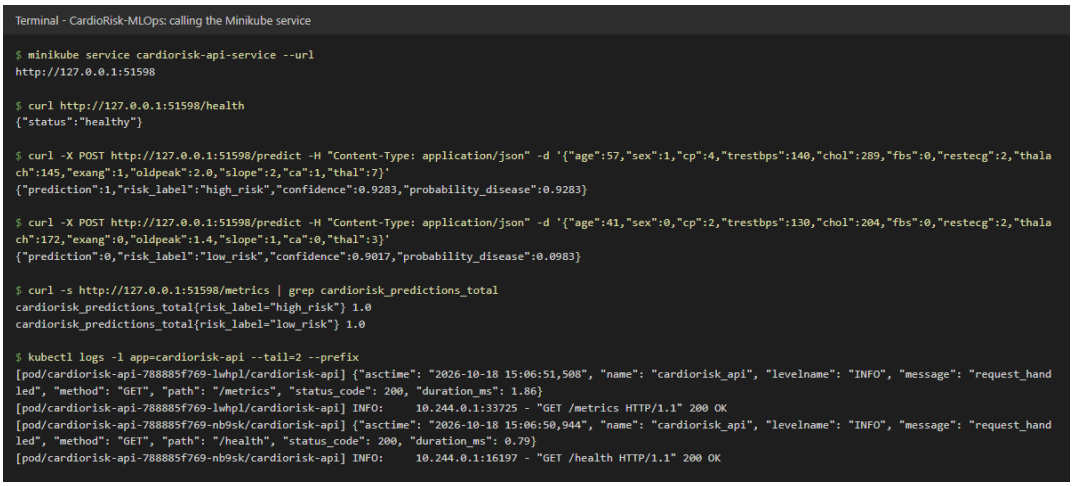
$ kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
deployment.apps/cardiorisk-api unchanged
service/cardiorisk-api-service unchanged

$ kubectl rollout status deployment/cardiorisk-api
deployment "cardiorisk-api" successfully rolled out

$ kubectl get deployments
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
cardiorisk-api   2/2     2            2           38s

$ kubectl get pods -l app=cardiorisk-api -o wide
NAME                              READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
cardiorisk-api-788885f769-lwhpl   1/1     Running   0          38s   10.244.0.4   minikube   <none>           <none>
cardiorisk-api-788885f769-nb9sk   1/1     Running   0          38s   10.244.0.3   minikube   <none>           <none>

$ kubectl get svc cardiorisk-api-service
NAME                    TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
cardiorisk-api-service   LoadBalancer   10.98.183.107   <pending>     80:32292/TCP   38s
```



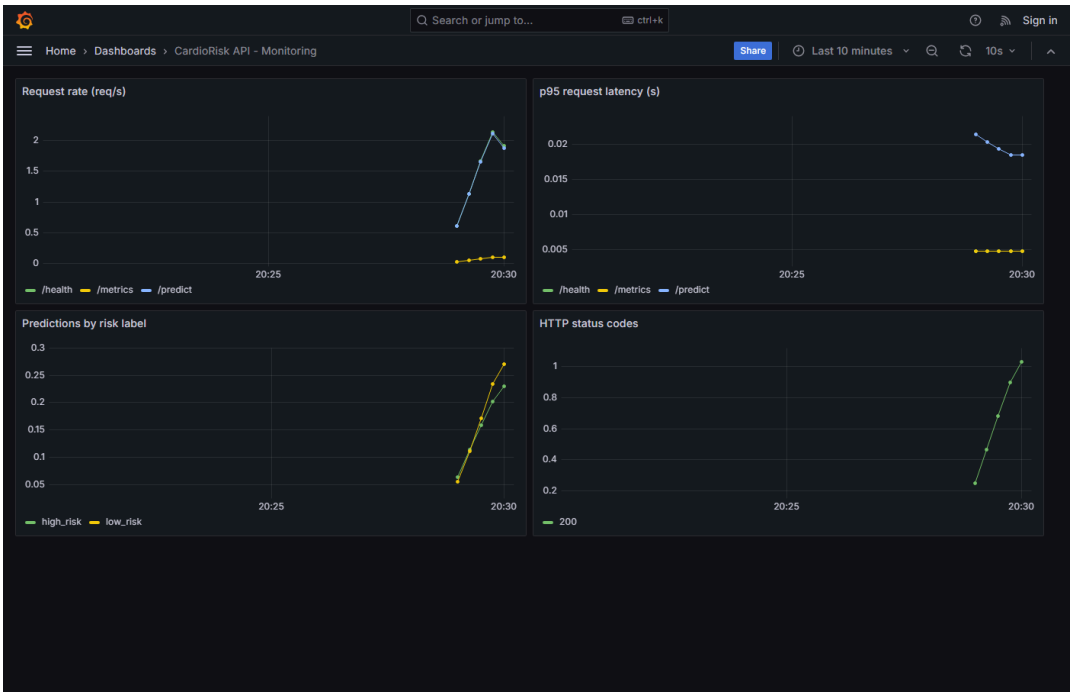
Both replicas came up `1/1 Running` and passed their readiness probes. The recent Minikube release uses containerd rather than Docker as its runtime, so `minikube docker-env` no longer applies; the locally built image is copied into the cluster with `minikube image load` instead, which is why `imagePullPolicy: IfNotPresent` is set. On the Windows Docker driver, `minikube service --url` opens a tunnel to `127.0.0.1`, and both a high-risk and a low-risk sample were scored through it.

10. Task 8 - Monitoring & Logging

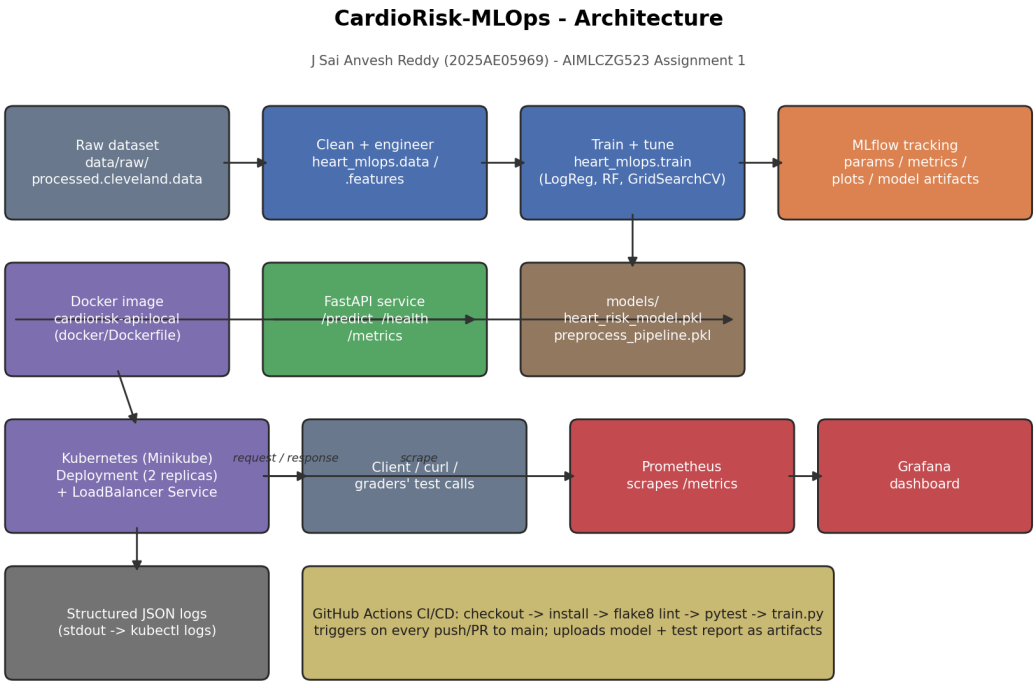
Request logging: every HTTP request is logged as structured JSON to stdout (`src/api/logging_config.py`, via a FastAPI middleware in `src/api/main.py`) - method, path, status code, and latency in ms. Because it goes to stdout, `docker logs` / `kubectl logs` capture it with no extra plumbing.

Metrics: `src/api/metrics.py` defines three Prometheus metrics exposed at `/metrics`: `cardiorisk_requests_total{method, path, status_code}` (Counter), `cardiorisk_request_latency_seconds{path}` (Histogram) and `cardiorisk_predictions_total{risk_label}` (Counter, updated in `/predict`).

Stack: `docker-compose.yml` runs the API with Prometheus (scraping `/metrics` every 10s, `monitoring/prometheus.yml`) and Grafana, pre-provisioned with the Prometheus datasource and a 4-panel "CardioRisk API - Monitoring" dashboard (request rate, p95 latency, predictions by risk label, HTTP status codes). `docker compose up --build` starts it all - Prometheus on `:9090`, Grafana on `:3000`.



11. Architecture Diagram



The raw Cleveland CSV is cleaned and feature-engineered, fed into grid-searched model training that logs everything to MLflow and saves the winning pipeline to `models/`. That pipeline is baked into a Docker image serving a FastAPI app, deployed to Minikube behind a LoadBalancer Service with 2 replicas. Prometheus scrapes `/metrics` and Grafana visualizes it; structured JSON logs flow to stdout. A separate GitHub Actions pipeline lints, tests, and re-trains on every push, independent of the runtime deployment path.

12. Setup / Install Instructions

```
python -m venv .venv && source .venv/Scripts/activate      # Python 3.11
pip install -r requirements.txt && pip install -e .
python -m heart_mlops.data                                 # data
jupyter notebook notebooks/01_eda.ipynb                    # EDA
python -m heart_mlops.train                                # train + log to MLflow
mlflow ui --backend-store-uri file:./mlruns                # view runs
pytest tests/ -v                                           # tests
uvicorn api.main:app --app-dir src --port 8000             # API, no Docker
docker build -f docker/Dockerfile -t cardiorisk-api:local .   # container
docker run -p 8000:8000 cardiorisk-api:local
docker compose up --build                                  # + Prometheus/Grafana
# Minikube deployment: see k8s/README.md
```

Deployed API access: local-only (Minikube) - no public cloud endpoint was provisioned. Follow `k8s/README.md`, then call the URL printed by `minikube service cardiorisk-api-service --url`. Full details for every step are in the top-level `README.md`.

13. Conclusion and Future Work

This assignment's main lesson, for me, wasn't the classifier itself (a 303-row tabular dataset doesn't leave much room for modelling heroics) - it was how much infrastructure sits around a genuinely simple model once you commit to tracking, testing, containerizing, orchestrating, and monitoring it properly. Logistic Regression beating Random Forest here is itself a small lesson in matching model complexity to data size rather than reaching for the fancier algorithm by default.

Given more time, the natural next steps would be: a held-out "shadow" evaluation set collected after deployment to catch data drift, a Helm chart instead of raw manifests, and promoting MLflow runs through a proper Model Registry (`Staging` -> `Production`) instead of the file-based best-model selection used here.

14. Repository Link

GitHub repository: **https://github.com/SaiAnveshReddy/MLOps-Assignment**